Off the Bench

ANSWER KEY (full solutions). Build an AI-assisted sports technique analyzer with Python, MediaPipe and Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

Welcome: what you are building . . . 3

How to use this tutorial . . . 3

The roadmap . . . 3

Before you start . . . 5

Set up your project . . . 5

Read the "Setup status" line . . . 5

Optional: AI feedback and privacy . . . 5

The big picture . . . 6

Build it, step by step . . . 7

Step 1: Run the starter and learn how images work . . . 7

Step 2: Choose a sport and a skill level . . . 9

Step 3: Display measurements and suggestions nicely . . . 12

Step 4: Detect landmarks with MediaPipe . . . 14

Step 5: Measure body angles . . . 16

Step 6: Coaching suggestions . . . 20

Step 7: Optional AI feedback . . . 22

Step 8: Optional extensions . . . 24

What you learned . . . 24

Welcome: what you are building

Off the Bench is a web app that looks at a photo of an athlete and gives technique ideas. You will be able to:

1. Choose a **sport** and a **skill level**.
2. Upload a photo of a movement (a jump shot, a kick, a dance pose).
3. Let a computer-vision model find the **body landmarks** (shoulders, elbows, knees, ...) and draw a skeleton on your photo.
4. Measure **joint angles** from those landmarks, such as "the right elbow is about 90 degrees".
5. Turn the measurements into **coaching suggestions** that depend on the sport and level.
6. Optionally ask an **AI model** for extra tips (only if you give consent to send your photo).

The folder you were given is a **scaffold**. The app already launches, but the important functions are simple placeholders marked `TODO`. This tutorial walks you through every `TODO`, in the order you should do them. For each one you will find:

- **Goal**: what the function should do, in plain English.
- **Ideas you need**: the small bit of Python being practiced.
- **Do it**: step-by-step instructions with code.
- **Check it**: how to run it and what you should see.

> **WARNING**
> Honesty matters in this project. One photo is one instant. It cannot prove a whole movement is right or wrong, the angles are approximate, and they change a lot with the camera angle. Your app should always say so. You will see this idea again and again.

How to use this tutorial

- Keep this PDF open next to your editor. Work through the steps **in order**. Each step builds on the last.
- Code shown with a file name above it (for example `sports.py`) belongs in that file.
- Whenever you see **Checkpoint**, stop and run the app (or the test shown) before moving on.
- Stuck? Read the **Stuck?** boxes. They list the mistakes students make most often.
- After you edit a Python file, **stop the app (`Ctrl + C`) and run `python app.py` again**.

The roadmap

Step	File	What you build
1	`app.py`	Run the starter, learn how images work, add the color-channel count (TODO 1)
2	`sports.py`	Add a sport, write photo tips and level guidance (TODOs 1-3)
3	`app.py`	Nicer measurement and suggestion display (TODOs 2-3)
4	`pose_analysis.py`	Download the model, detect landmarks, practice TODOs
5	`feedback.py`	Measure angles and classify them (TODOs 1-5)

Step	File	What you build
6	`feedback.py`	Level-aware coaching and drills (TODOs 6-7)
7	`ai_helper.py`	The AI prompt and answer formatting (TODOs 1-3)
8	any	Optional extensions

Before you start

Set up your project

Open a terminal inside the `off_the_bench` folder and run these commands one at a time.

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env             # Windows: copy .env.example .env
python app.py
```

What each line does:

- `python -m venv .venv` creates a private "box" of Python packages just for this project.
- `source .venv/bin/activate` steps into that box. You should see `(.venv)` at the start of your terminal line. **Do this every time you open a new terminal.**
- `pip install -r requirements.txt` installs Gradio (the web page), NumPy (image numbers), Pillow (image files), MediaPipe (pose detection) and python-dotenv (reads your `.env` file).
- `cp .env.example .env` makes your private settings file. You can leave the key blank for now.
- `python app.py` starts the app. Open the address it prints (usually `http://127.0.0.1:7860`).

> **WARNING**
> MediaPipe works with **Python 3.9 to 3.12**. If `pip install` complains about MediaPipe, check your version with `python --version` . The app **still launches without MediaPipe**: it just turns pose detection off and says so in the status line. You can finish Steps 1 to 3 and Steps 5 to 7 using the fake-landmark tests in this tutorial even if MediaPipe will not install.

Read the "Setup status" line

At the top of the page the app prints a line like this:

```
Setup status: MediaPipe is not installed (pip install mediapipe). Pose detection is off. | AI
    feedback is off: no ANTHROPIC_API_KEY found (see .env.example).
```

That line is your dashboard. It has two halves: **pose detection** (MediaPipe plus its model file) and **AI feedback** (an API key). Each half can be on or off independently. When both are on, you will see `Pose detection is ready.` and `AI feedback is available.`

Optional: AI feedback and privacy

AI feedback needs an Anthropic API key (your teacher may give you one) and the extra package: `pip install anthropic` . Put the key in `.env` as `ANTHROPIC_API_KEY=...` .

> **WARNING**
> Never paste your key into a Python file, share it, or commit `.env` to GitHub. Photos of people are personal. This app sends your photo to the AI **only if the user ticks the consent checkbox**. Without it, only the text measurements are sent.

The big picture

```
choose sport -> upload image -> detect landmarks -> measure angles -> coach -> (optional) AI
  sports.py       app.py        pose_analysis.py     feedback.py     feedback.py  ai_helper.py
```

File	Job
`app.py`	The screen, and the `analyze` function that connects every stage
`sports.py`	The list of sports, photo tips, and what to tell each skill level
`pose_analysis.py`	A wrapper around MediaPipe: finds landmarks, draws the skeleton, measures angles. It is kept separate on purpose.
`feedback.py`	**Measurements** (facts from the photo) and **coaching** (opinions based on facts)
`ai_helper.py`	The optional AI prompt and request, with the consent check

Keep one rule in your head all project long: **measurements are facts about the photo; coaching is your interpretation of them. Keep them separate.** `feedback.py` has one function for each.

Build it, step by step

Step 1: Run the starter and learn how images work

File: `app.py` **Function:** `describe_image(image)` **TODO 1**

Goal: understand the flow of the app, what an image looks like to Python, and report the number of color channels of an upload.

Ideas you need: NumPy arrays, `.shape`, f-strings.

1a. Click around

Run `python app.py`. Choose a sport and watch the text under it change (the `show_sport_info` function in `app.py` does that). Upload any photo, **leave "Detect body landmarks" unticked**, and click **Analyze**. You will see "Image received: ... pixels.", a short list of suggestions, and a disclaimer.

1b. How Python sees a picture

Read the comment above `describe_image`. When you drop a file into `gr.Image(type="numpy")`, Gradio hands your function a **NumPy array**: a grid of numbers with the shape **(height, width, 3)**.

- The first two numbers are the rows and columns of pixels.
- The `3` is the number of **color channels**: red, green and blue.
- Every value is between 0 (none of that color) and 255 (full color).

You can see this for yourself in a Python shell:

```
python -c "import numpy as np; a = np.zeros((480, 640, 3), dtype=np.uint8); print(a.shape, a[0,
    0])"
```

```
(480, 640, 3) [0 0 0]
```

A 640 by 480 photo is an array with 480 rows (height comes first!) and 640 columns. The pixel in the top-left corner is three numbers, here `[0 0 0]`, which is black.

> **CONCEPT**
> In `describe_image`, the line `height, width = image.shape[:2]` takes only the first two items of the shape. `[:2]` means "from the start up to, but not including, position 2". That is why height comes before width in the message-building line below it.

TODO 1: report the color channels

`image.shape[2]` is the number of channels. Add it to the message. Be careful: a *grayscale* image has only two numbers in its shape, so `image.shape[2]` would crash with `IndexError`. `image.ndim` tells you how many numbers are in the shape.

```
channels = image.shape[2] if image.ndim == 3 else 1
```

That one line is a **conditional expression**: "use `image.shape[2]` if the image has 3 dimensions, otherwise use 1".

The starter question asked: *why might a PNG have 4 channels?* The fourth channel is **alpha**, which stores transparency. Photos saved from some phones and screenshots are often PNGs with alpha. (The landmark detector later drops the alpha channel for you. Look for `image[:, :, :3]` in `pose_analysis.py`.)

Finished function

app.py

```
def describe_image(image):
    """Return a short description of the uploaded image, or an error message."""
    if image is None:
        return "No image uploaded yet."
    height, width = image.shape[:2]
    channels = image.shape[2] if image.ndim == 3 else 1
    return f"Image received: {width} x {height} pixels, {channels} color channel(s)."
```

CHECKPOINT

Restart and analyze a photo. The status box should read `Image received: 640 x 480 pixels, 3 color channel(s).` (with your photo's numbers). Upload a PNG with transparency and check that you see 4. Try it from the terminal too: `describe_image(np.zeros((480, 640, 4), dtype=np.uint8))` gives `Image received: 640 x 480 pixels, 4 color channel(s).`

STUCK?

`IndexError: tuple index out of range` means you used `image.shape[2]` on a grayscale image. Use the `if image.ndim == 3 else 1` version. If nothing changes in the browser, you did not restart the app.

Step 2: Choose a sport and a skill level

File: `sports.py` **TODOs 1, 2, 3**

Goal: add a new sport, write a photo-taking tip for every sport, and describe how feedback should change for each skill level.

Ideas you need: dictionaries and lists, `if / elif / else`.

Read the data

At the top of `sports.py`, `SPORTS` is a **dictionary** that maps each sport to a **list** of skills. `get_sport_names()` returns just the keys, which is how the dropdown in `app.py` gets its choices:

```
sport = gr.Dropdown(sports.get_sport_names(), value="basketball", label="Sport")
```

TODO 1: add one more sport

Add a new key-and-list pair. We will add tennis. Don't forget the comma after the entry above it.

`sports.py`

```
SPORTS = {
    "basketball": ["jump shot (release)", "free throw stance", "dribbling stance", "layup
        takeoff"],
    "soccer": ["instep kick (plant foot)", "throw-in", "goalkeeper ready stance", "header setup"],
    "volleyball": ["overhead set", "forearm pass (platform)", "spike approach", "serve toss"],
    "dance": ["arabesque", "plie", "arm positions", "balance pose"],
    "tennis": ["serve toss", "forehand contact", "backhand preparation", "ready position"],
}
```

Does the rest of the app still work? The starter asks you to find out, so test it. Restart the app and open the sport dropdown. **Tennis appears immediately**, because the dropdown is built from the dictionary's keys and nothing else needed to change. Select it: you will see its example skills. The photo tip underneath, however, is the generic fallback, `Make sure your whole body is visible and well lit.`, because `get_sport_instructions` has no tennis branch yet and falls into its `else`. **So the app does not crash, but it is not good yet.** The next TODO fixes it.

> **WHY**
> A good `else` branch is a safety net. It let a brand-new sport work with zero crashes. But "does not crash" is not the same as "works well". Always test the new case yourself.

TODO 2: photo tips for every sport

Camera angle changes every measurement later, so the tips matter. Basketball is done as an example. Write a branch for each other sport that says **which body parts must be visible** and **which side the camera should be on**. Add an `elif sport == "tennis":` branch too.

sports.py

```
def get_sport_instructions(sport):
    """
    Return a short photo-taking tip for the chosen sport.

    Good tips matter because camera angle affects every measurement later!
    """
    if sport == "basketball":
        return ("Photograph from the side at about chest height. Make sure your shooting "
                "arm, shoulder and hip are all in the frame.")
    elif sport == "soccer":
        return ("Photograph from the side, at about knee height, so the kicking leg, the "
                "plant-foot knee and the ankle are all in the frame.")
    elif sport == "tennis":
        return ("Photograph from the side or slightly behind, at about waist height. "
                "Keep the hitting arm, shoulder, hip and both knees in the frame.")
    elif sport == "volleyball":
        return ("Photograph from the side at about chest height. Make sure both arms, "
                "both shoulders and both knees are in the frame.")
    elif sport == "dance":
        return ("Photograph from the front or the side against a plain background. Wear "
                "fitted clothing and keep your whole body, head to toes, in the frame.")
    else:
        return "Make sure your whole body is visible and well lit."
```

TODO 3: guidance for each skill level

Should a beginner get one simple tip or five detailed ones? Beginner is written for you. Finish intermediate and advanced.

sports.py

```
def get_level_guidance(level):
    """
    Return a sentence describing how feedback should be tailored to a level.
    Used by feedback.py and ai_helper.py.
    """
    if level == "beginner":
        return "Give one or two simple, encouraging tips. Avoid jargon."
    elif level == "intermediate":
        return "Give two or three tips that mention specific body positions, with a short reason
            for each."
    elif level == "advanced":
        return "Give detailed, technical tips about small adjustments, and mention trade-offs."
    else:
        return "Give general feedback."
```

NOTE

These sentences are written like *instructions to a coach* ("Give two or three tips..."). They are used two ways: `build_suggestions` shows them to the user, and `ai_helper.py` passes them to the AI to control how much it says. They read most naturally as instructions to the AI. If you would like them to read better on screen, reword them so they work for both readers.

CHECKPOINT

Restart. For each of the five sports, select it and read the photo tip. None should say `TODO`. Then upload any photo, choose a level, click Analyze (leave pose off), and check that the suggestions change when you change the level. In a terminal you can test directly: `python -c "import sports; print(sports.get_level_guidance('advanced'))"`.

STUCK?
A `SyntaxError` in `SPORTS` is almost always a missing comma between two entries. Remember that `elif` needs a colon at the end, and compare with `==` (two equals signs), not `=`.

Step 3: Display measurements and suggestions nicely

File: `app.py` **TODOs 2 and 3**

Goal: the starter prints measurements and suggestions as plain bullet lists. Make the display friendlier by moving that work into two small helper functions.

Ideas you need: `for` loops, f-strings, `round` , `enumerate` , writing a function.

Read the main handler first

`analyze(...)` is the function Gradio calls when you click **Analyze**. Read it top to bottom and match it to the flow diagram: **validate the upload, optionally detect the pose, measure, build coaching suggestions, optionally ask the AI**. It returns one value for each output component, in the same order as the `outputs=[...]` list at the bottom of the file.

> **CONCEPT**
> `measurements` is a **list of dictionaries**, for example `[{"name": "Right elbow angle", "value": 142.0, "label": "bent"}]` . `value` is `None` when the angle could not be measured. You will create those dictionaries in Step 5. For now, just know their shape.

TODO 2: improve how measurements are displayed

The starter loops over the measurements inline. Move it into a function so `analyze` stays short. Improve it too: round the angle, and show a friendly message when the value is `None` (rather than `n/a`).

app.py

```
def format_measurements(measurements):
    """Turn the measurement dictionaries into a Markdown list."""
    lines = []
    for m in measurements:
        if m["value"] is None:
            lines.append(f"- **{m['name']}:** could not measure (landmark hidden)")
        else:
            lines.append(f"- **{m['name']}:** about {round(m['value'])} degrees ({m['label']})")
    return "\n".join(lines)
```

Then replace the whole `lines = []` ... `measurements_text = ...` block inside `analyze` with one line:

```
measurements_text = format_measurements(measurements)
```

`round(142.0)` gives `142` , so we drop the decimals. The `**...**` makes the name bold in the Markdown component.

TODO 3: format the suggestions

Make the suggestions a **numbered** list instead of bullets. `enumerate(suggestions, start=1)` gives you `(1, first)` , `(2, second)` and so on, which is exactly what a numbered list needs.

app.py

```
def format_suggestions(suggestions):
    """Turn the suggestion strings into a numbered Markdown list plus the disclaimer."""
    numbered = [f"{number}. {text}" for number, text in enumerate(suggestions, start=1)]
    return "\n".join(numbered) + f"\n\n_{feedback.DISCLAIMER}_"
```

Replace the `suggestions_text = ...` line in `analyze` with:

```
suggestions_text = format_suggestions(suggestions)
```

Note that the disclaimer stays attached: `feedback.DISCLAIMER` must always be visible.

CHECKPOINT

Restart, upload a photo with pose off and click Analyze. The suggestions are now numbered, followed by the italic disclaimer. You cannot see the measurement list yet (it needs pose detection), so test the function directly, from inside `off_the_bench` : `python -c "from app import format_measurements as f; print(f([{'name':'Right elbow angle','value':141.6,'label':'bent'},{'name':'Left elbow angle','value':None,'label':'x'}]))"` You should see `- **Right elbow angle:** about 142 degrees (bent)` and `- **Left elbow angle:** could not measure (landmark hidden)` .

STUCK?

If the app breaks on launch, you probably pasted a function in the middle of another one. Define both helpers at the top level of the file (no indent), above `analyze` . A `NameError: name 'format_measurements' is not defined` means the helper is defined *below* where it is called, or never saved.

Step 4: Detect landmarks with MediaPipe

File: `pose_analysis.py` **Feature 4 setup and the two practice TODOs**

Goal: turn on pose detection, see the skeleton on a real photo, and write a tiny geometry helper.

Ideas you need: running a script with a flag, dictionaries of dictionaries, coordinates.

What is a landmark?

MediaPipe is a free library from Google. Given a photo, a pre-trained model guesses where **33 body points** are: nose, shoulders, elbows, wrists, hips, knees, ankles and more. You do not train anything. You get coordinates and then use ordinary Python (`if` statements and a little geometry) to measure things.

Each landmark, in the dictionary that `detect_pose` returns, looks like this:

```
"right_elbow": {"x": 0.43, "y": 0.51, "z": -0.12, "visibility": 0.98,
                "x_px": 275.2, "y_px": 244.8}
```

(The numbers here are only an example shape. Yours will differ.)

- `x` and `y` are positions as a **fraction** of the picture (0.0 is left or top, 1.0 is right or bottom).
- `x_px` and `y_px` are the same position in **pixels**. The code uses these for measuring.
- `visibility` says how confident MediaPipe is that the point is really visible (0 to 1).
- "left" and "right" mean the **person's** left and right, not the image's.

> **CONCEPT**
> In pictures, **y grows downward**. A point near the top of the photo has a *small* `y_px` . This surprises almost everyone the first time, and it matters whenever you ask "is the wrist above the shoulder?"

4a. Download the model and check the setup

MediaPipe needs a model file (about 6 MB). From inside `off_the_bench` , with your virtual environment active, run:

```
python pose_analysis.py --download
```

It saves `models/pose_landmarker_lite.task` . Next, ask the file for its own status:

```
python pose_analysis.py
```

If everything is installed, it prints `Pose detection is ready.` If not, it tells you what is missing: either `MediaPipe is not installed (pip install mediapipe). Pose detection is off.` or `Pose model file missing. Run: python pose_analysis.py --download` . The function behind this is `check_pose_setup()` . It never raises an error, so the app can always launch. That is a good habit.

4b. Turn it on

Restart the app. The setup status should now say `Pose detection is ready.` and the checkbox **Detect body landmarks** should be ticked by default (`value=pose_ready`). Upload a clear, whole-body photo and click **Analyze**.

What to expect, if MediaPipe is working: the "Detected landmarks" picture shows red dots and cyan lines drawn over the person, the status says `Pose: Pose detected. Landmark positions are estimates and`

`may be off.` , and a measurement list appears (only one line for now, until Step 5). If the photo has no clear person you will see `No person/pose detected. Try a clearer photo with your whole body visible and good lighting.`

> **STUCK?**
> `Pose model file missing` means you did not run the download command (or ran it in a different folder). If `pip install mediapipe` fails, check `python --version` is between 3.9 and 3.12. A network error during the download usually means you need to connect to the internet or a school proxy is blocking it.

Read `detect_pose` (no coding)

Read the function in the order it works. It (1) checks the image is a usable color array, (2) checks MediaPipe and the model exist, (3) runs the landmarker on one still photo, (4) converts MediaPipe's output into the simple dictionary above, and (5) draws the skeleton. Its **return value is always a dictionary with the same four keys** (`found` , `status` , `landmarks` , `annotated_image`), even on failure. That is why `analyze` can safely do `pose["status"]` without checking anything first.

Practice TODO 1: write `get_midpoint`

At the bottom of the helper section, add a function that returns the point halfway between two `(x, y)` points. Average the x values and average the y values. (Could the midpoint of the two hips be useful? It is a good estimate of the body's center.)

`pose_analysis.py`

```
def get_midpoint(p1, p2):
    """Return the (x, y) point halfway between two (x, y) points."""
    return ((p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2)
```

Test it:

```
python -c "from pose_analysis import get_midpoint; print(get_midpoint((0, 0), (10, 20)))"
```

```
(5.0, 10.0)
```

Practice TODO 2: look at a real landmark

This one is an experiment, not code to keep. With a test photo (call it `photo.jpg`) in the folder, run:

```
python -c "import numpy as np; from PIL import Image; import pose_analysis as pa; img =
    np.array(Image.open('photo.jpg').convert('RGB')); r = pa.detect_pose(img);
        print(r['status']); print(r['landmarks']['right_wrist'])"
```

Do it twice, once with the wrist **low** and once **raised above your head**. Compare the `y_px` values. Is `y_px` bigger or smaller when the wrist is higher in the picture? (Remember: y grows downward, so it should be **smaller**.) Write your answer in a comment in your notes.

> **CHECKPOINT**
> (1) `python pose_analysis.py` says what is ready or missing. (2) With MediaPipe working, a photo of a standing person shows a skeleton overlay. (3) `get_midpoint((0, 0), (10, 20))` returns `(5.0, 10.0)` . If you cannot get MediaPipe installed, do (3) and continue: the next steps can be tested with fake landmarks.

Step 5: Measure body angles

File: `feedback.py` **TODOs 1, 2, 3, 4, 5**

Goal: turn landmarks into **measurements**: a list of joint angles, each with a plain-language label.

Ideas you need: functions, lists of tuples, `for` loops, `None`, `if / elif / else`.

The angle math, conceptually

The helper `calculate_angle(a, b, c)` in `pose_analysis.py` is already written. It returns the angle at the middle point `b`, made by the lines from `b` to `a` and from `b` to `c`. For an elbow, `a` is the shoulder, `b` is the elbow, `c` is the wrist.

Inside, each line becomes an **arrow** (a vector). The code finds how much the two arrows point in the same direction and converts that to degrees. You do not need to follow the trigonometry, but you should be able to *read* results:

```
python -c "from pose_analysis import calculate_angle as c; print(c((0, 0), (0, 10), (10, 10)));
    print(c((0, 0), (0, 10), (0, 20)))"
```

```
90.0
180.0
```

Draw the first one on paper. Shoulder at (0, 0), elbow at (0, 10), wrist at (10, 10): the upper arm goes straight down and the forearm goes sideways, a perfect **right angle (90 degrees)**. In the second, shoulder, elbow and wrist all sit on one line, a straight arm, **180 degrees**. A small angle means a tightly bent joint.

> **WARNING**
> The angle is measured **in the 2D photo**. If the arm points toward or away from the camera, the real angle can look very different. This is why the photo tips in Step 2 matter, and why the app is full of disclaimers.

Read `measure_joint_angle`

feedback.py

```python
def measure_joint_angle(landmarks, first, middle, last):
    """
    Angle at `middle` between `first` and `last` (all landmark names), or None
    if any landmark is missing / not visible.
    Example: measure_joint_angle(landmarks, "right_shoulder", "right_elbow", "right_wrist")
    """
    points = [get_point(landmarks, name) for name in (first, middle, last)]
    if None in points:
        return None
    return calculate_angle(*points)
```

It asks `get_point` for three landmarks. `get_point` returns `None` when a landmark is missing **or not visible enough** (visibility below 0.5), so the function checks `if None in points` and gives up gracefully, returning `None`. **Always check for `None` before using a landmark.**

Build a fake body so you can test (no photo needed)

Make a file called `fake_landmarks.py` in the `off_the_bench` folder. It builds landmarks by hand, in the same shape as `detect_pose`'s output, so you can test your rules without a photo or MediaPipe:

fake_landmarks.py

```
from feedback import analyze_pose


def lm(x, y, visibility=0.9):
    return {"x": x / 100, "y": y / 100, "z": 0, "visibility": visibility,
            "x_px": x, "y_px": y}


fake = {
    "right_shoulder": lm(0, 0),  "right_elbow": lm(0, 10),  "right_wrist": lm(10, 10),
    "left_shoulder": lm(50, 0),  "left_elbow": lm(50, 10),  "left_wrist": lm(50, 20),
    "right_hip": lm(0, 30),      "right_knee": lm(10, 40),  "right_ankle": lm(0, 40),
    "left_hip": lm(50, 30),      "left_knee": lm(50, 40),   "left_ankle": lm(55, 50),
}

for sport in ["basketball", "soccer", "volleyball"]:
    print(sport)
    for m in analyze_pose(fake, sport):
        print("  ", m["name"], m["value"], m["label"])

fake["right_wrist"] = lm(10, 10, visibility=0.1)      # pretend the wrist is hidden
print(analyze_pose(fake, "basketball")[0])
```

Run it with `python fake_landmarks.py`. Right now it prints one right-elbow line for every sport and a label that says `not classified yet (finish TODO 3)`. By the end of this step the output will be rich.

TODO 3: classify an elbow angle

(The starter numbers its TODOs in a different order than the file, so we follow the order you will actually write code.) `classify_elbow_angle` turns a number into words. The starter suggests: above about 160 means "nearly straight", about 70 to 160 means "bent", and below 70 means "tightly bent". Use `if / elif / else`.

feedback.py

```
def classify_elbow_angle(angle):
    """Turn an elbow angle into a plain-language label."""
    if angle > 160:
        return "nearly straight"
    elif angle >= 70:
        return "bent"
    else:
        return "tightly bent"
```

Think about the order of the checks. Python tests from the top and stops at the first one that is true, so `> 160` goes first. **Pick thresholds you can defend**, and try changing them against real photos. Add a knee version too (knees bend differently from elbows):

feedback.py

```
def classify_knee_angle(angle):
    """Turn a knee angle into a plain-language label."""
    if angle > 165:
        return "nearly straight"
    elif angle >= 110:
        return "slightly bent"
    else:
        return "deeply bent"
```

TODO 1: find the shoulder, elbow and wrist (right AND left)

The starter measures only the right arm. The landmark names for the left arm are `left_shoulder`, `left_elbow`, `left_wrist`. Instead of copying the same lines again and again, describe every joint **once** in a list of tuples: `(display name, first landmark, middle landmark, last landmark, classifier function)`. A function can be stored in a variable and used later, as you can see in the last item of each tuple.

feedback.py

```
ELBOWS = [
    ("Right elbow angle", "right_shoulder", "right_elbow", "right_wrist", classify_elbow_angle),
    ("Left elbow angle", "left_shoulder", "left_elbow", "left_wrist", classify_elbow_angle),
]
```

TODO 2: calculate the angle, and add the knees

Calculating the angle is the job of `measure_joint_angle` (already done). The knee needs the hip, knee and ankle:

feedback.py

```
KNEES = [
    ("Right knee angle", "right_hip", "right_knee", "right_ankle", classify_knee_angle),
    ("Left knee angle", "left_hip", "left_knee", "left_ankle", classify_knee_angle),
]
```

TODO 5: choose which joints matter for each sport

A basketball shot cares about the elbow; a soccer kick about the plant-foot knee. Use `if / elif / else` on `sport` to choose which list of joints to measure. Lists can be joined with `+`: `ELBOWS + KNEES` is a list with all four.

TODO 4: handle a hidden landmark gracefully

Now write `analyze_pose`. For each joint, measure the angle. If the result is `None` (landmark hidden), give a friendly label instead of crashing on `classify(angle)`. Put all four TODOs together:

feedback.py

```
def analyze_pose(landmarks, sport):
    """
    Return a list of measurement dictionaries, e.g.
      [{"name": "Right elbow angle", "value": 142.0, "label": "bent"}]
    `value` is None when the measurement could not be made.
    """
    measurements = []

    # Which joints matter depends on the sport.
    if sport == "basketball":
        joints = ELBOWS
    elif sport == "soccer":
        joints = KNEES
    else:  # volleyball, tennis, dance, anything new: look at arms and legs
        joints = ELBOWS + KNEES

    for name, first, middle, last, classify in joints:
        angle = measure_joint_angle(landmarks, first, middle, last)
        if angle is None:
            label = "could not measure (landmark hidden)"
        else:
            label = classify(angle)
        measurements.append({"name": name, "value": angle, "label": label})

    return measurements
```

The key line is `label = classify(angle)` : `classify` is whichever function the tuple held, so the same loop labels elbows and knees.

CHECKPOINT

Run `python fake_landmarks.py` . You should see exactly this:

```
basketball
   Right elbow angle 90.0 bent
   Left elbow angle 180.0 nearly straight
soccer
   Right knee angle 45.00000000000001 deeply bent
   Left knee angle 153.434948822922 slightly bent
volleyball
   Right elbow angle 90.0 bent
   Left elbow angle 180.0 nearly straight
   Right knee angle 45.00000000000001 deeply bent
   Left knee angle 153.434948822922 slightly bent
{'name': 'Right elbow angle', 'value': None, 'label': 'could not measure (landmark hidden)'}
```

NOTE

`45.00000000000001` is normal. Computers store decimals approximately, which is one more reason the app rounds before showing numbers. The last line proves the hidden-landmark case does not crash.

STUCK?

`TypeError: 'NoneType' object is not callable` , or a crash on `None` , means you called `classify(angle)` without the `if angle is None` check. `ValueError: too many values to unpack` means a tuple in your list has the wrong number of items (it needs five). If every sport gives the same joints, check you used `elif` and compared with `==` .

Step 6: Coaching suggestions

File: `feedback.py` **TODOs 6 and 7**

Goal: turn measurements into advice that depends on the **label** and the athlete's **level**, and add a drill for the sport.

Ideas you need: `for` loops, `if / elif / else`, dictionaries, `in`.

> **CONCEPT**
> This is where facts become opinions. A measurement ("the elbow is 90 degrees in this photo") is a fact. "Try more bend" is an interpretation. Use gentle words: **might**, **try**, **notice**. Avoid **wrong** and **correct**, because one photo cannot know.

TODO 6: a suggestion for each measurement

First write a small function that makes **one** sentence for **one** measurement. Beginners get gentle and simple language; advanced athletes get more specific:

`feedback.py`

```
def suggestion_for(measurement, level):
    """One coaching sentence for one measurement, worded for the athlete's level."""
    name = measurement["name"].lower()
    label = measurement["label"]
    if level == "beginner":
        return f"Your {name} looks {label} in this photo. Notice how that feels, and try small
            changes."
    elif level == "intermediate":
        return (f"Your {name} looks {label} (about {round(measurement['value'])} degrees). "
                "You might compare it with a second photo from the same side.")
    else:
        return (f"Your {name} is {label} at roughly {round(measurement['value'])} degrees "
                "in this frame. It might be worth checking how it changes through the whole
                    movement.")
```

The `name` and `label` are placed inside the sentence with f-strings. Notice that `round(measurement['value'])` is only used in the levels that show the number.

Now use it in `build_suggestions`: loop over `measurements`, skip any whose `value` is `None` (nothing to say about a joint we could not see), and add the sentence to the list.

TODO 7: a drill for each sport

Make a dictionary from sport to a drill, and add the matching one:

feedback.py

```
DRILLS = {
    "basketball": "Try a wall shooting drill: stand close to a wall and focus on a smooth elbow
        and wrist follow-through.",
    "soccer": "Try a plant-foot drill: place your plant foot beside a marker on 10 slow kicks and
        notice your knee bend.",
    "volleyball": "Try 20 slow overhead sets against a wall, keeping your elbows bent and hands
        high.",
    "dance": "Practice the pose at a mirror, holding for five slow counts and checking your leg
        lines.",
    "tennis": "Shadow-swing slowly in front of a mirror, pausing at contact to check your elbow
        and knees.",
}
```

`if sport in DRILLS:` is the safe way to check that a key exists. A sport without a drill (a brand-new one, say) is simply skipped.

Finished function

feedback.py

```
def build_suggestions(sport, level, measurements):
    """
    Return a list of suggestion strings for the user.
    These are interpretations of the measurements, not measurements themselves.
    """
    suggestions = [get_sport_instructions(sport), get_level_guidance(level)]

    for m in measurements:
        if m["value"] is not None:
            suggestions.append(suggestion_for(m, level))

    if sport in DRILLS:
        suggestions.append(DRILLS[sport])

    return suggestions
```

The first two entries (the photo tip and the level guidance) were already in the starter.

CHECKPOINT

Add these lines to the bottom of `fake_landmarks.py` and run it:

`from feedback import build_suggestions`

`for level in ["beginner", "intermediate", "advanced"]: print(level, build_suggestions("basketball", level, analyze_pose(fake, "basketball")))`

Each level should give a **different wording**, and the wrist-hidden measurement should get no sentence. Then run the real app with a photo and pose on: the suggestion list should include your measured sentences and a drill.

STUCK?

`KeyError: 'value'` means a measurement dictionary is missing a key (check `analyze_pose` appends `name`, `value` *and* `label`). A sentence that says "None degrees" means you did not skip `value is None`. Same suggestions for every level means you are not using the `level` argument in `suggestion_for`.

Step 7: Optional AI feedback

File: `ai_helper.py` **TODOs 1, 2, 3**

Goal: build a good prompt from your measurements, and display the AI's reply with a permanent warning label.

Ideas you need: building strings, loops, importing from another file.

Read the safety design first

Open `ai_helper.py` and notice three safeguards that are already in the starter. They are not decoration.

- `is_ai_available()` never raises. If there is no key or no `anthropic` package, the app keeps working.
- `get_ai_feedback(...)` only attaches the image `if image is not None and user_consented_to_send_image`. **No consent, no photo.**
- `AI_LABEL` says the output is "not verified biomechanical analysis". It must stay visible.

TODO 1: measurements as a text block

A prompt is just text. The AI knows nothing about your photo, so describe the measurements in words. Loop through them, **skip any whose value is** `None`, and join the lines. If there are none, say so.

TODO 2: a better prompt

A good prompt says: **who the user is, what data we have, what we want back, and what not to claim.** Import `get_level_guidance` at the top of the file (`from sports import get_level_guidance`), add that sentence, ask for 2 to 3 short tips, and tell the AI to say when it is unsure and not to claim certainty from one photo.

`ai_helper.py`

```
def build_prompt(sport, level, measurements):
    """
    Create the text we send to the AI.
    A good prompt says: who the user is, what data we have, what we want back,
    and what NOT to claim.
    """
    lines = []
    for m in measurements:
        if m["value"] is not None:
            lines.append(f"- {m['name']}: about {round(m['value'])} degrees ({m['label']})")
    measurement_text = "\n".join(lines) if lines else "(no measurements provided)"

    return (
        f"My sport is {sport} and my skill level is {level}. Here are approximate measurements "
        f"from one photo:\n{measurement_text}\n\n"
        f"{get_level_guidance(level)}\n"
        "Give 2-3 short technique tips. The angles come from a single 2D photo, so "
        "do not claim certainty. If you are unsure about something, say so."
    )
```

TODO 3: format the reply

Keep `AI_LABEL` visible, always. `---` draws a divider line in Markdown.

ai_helper.py

```
def format_ai_feedback(raw_text):
    """Format the AI's reply for display."""
    return f"{raw_text.strip()}\n\n---\n_{AI_LABEL}_"
```

CHECKPOINT

You do not need a key for this. Test the prompt directly: `python -c "from ai_helper import build_prompt; print(build_prompt('basketball', 'intermediate', [{'name':'Right elbow angle','value':90.0,'label':'bent'},{'name':'Left elbow angle','value':None,'label':'x'}]))"`

Expect the prompt to mention the right elbow, **not** the hidden one:

```
My sport is basketball and my skill level is intermediate. Here are approximate measurements from
    one photo:
- Right elbow angle: about 90 degrees (bent)

Give two or three tips that mention specific body positions, with a short reason for each.
Give 2–3 short technique tips. The angles come from a single 2D photo, so do not claim certainty.
    If you are unsure about something, say so.
```

NOTE

With no key, ticking "Also ask an AI model" shows `AI feedback is off: no ANTHROPIC_API_KEY found (see .env.example).` followed by the label. That is expected: the starter formats whatever text comes back, including that message.

Turn it on and run the full test plan

Add your key to `.env` (`ANTHROPIC_API_KEY=...` , no quotes and no spaces), run `pip install anthropic` , and restart. The status line should say `AI feedback is available.`

Test	What you should see
Click Analyze with no photo	"Please upload an image first."
Photo, pose off, AI off	Image size, suggestions, disclaimer
Photo, pose on	Skeleton image, a measurement list, level-aware suggestions
Same photo, change level	Different wording in the suggestions
Tick AI, consent **unticked**	AI answer based on the text measurements only (no photo sent)
Tick AI, consent **ticked**	AI answer that can also describe the photo
Any AI answer	`AI_LABEL` text is shown at the bottom

STUCK?

`The AI request failed: ...` : read the end of the message. Authentication errors mean a wrong key or a stray space. If it says `no ANTHROPIC_API_KEY found` , `.env` must be in the same folder as `app.py` and you must have restarted. If the answer ignores your photo, check the consent box is ticked.

Step 8: Optional extensions

Finished early? Each of these is a real project. Do them in any order.

1. **Video.** Read frames with OpenCV (`pip install opencv-python`), run `detect_pose` on each one, and record an angle per frame. A movement is a **sequence**, so the angle's change over time says much more than one photo. Plot it.
2. **Compare two photos.** Add a second `gr.Image` , measure both, and report the difference ("your elbow is 20 degrees more bent in the second photo").
3. **Draw the angles.** In `draw_landmarks` , use `ImageDraw` to write each measured angle next to its joint.
4. **Skill drills.** Choose drills from the measurements (for example, a straight-arm shooter gets a different drill from a tightly bent one), not just the sport.
5. **Model sizes.** Try the `full` or `heavy` MediaPipe pose model by changing `MODEL_URL` (and the file name) in `pose_analysis.py` . Compare speed and accuracy.
6. **Use the midpoint.** Use `get_midpoint` for the two hips and shoulders, and measure how much the torso leans.

What you learned

- How an **image becomes a NumPy array**, and why shape is (height, width, channels).
- What **pose landmarks** are, and why `visibility` and `None` checks protect your code.
- **Angle measurement** from three points, and why a 2D photo limits it.
- `if / elif / else` , dictionaries, loops, and storing functions in a list of tuples.
- The difference between **facts** (measurements) and **opinions** (coaching), and why honest labels, disclaimers and **consent** are part of good software.
- Testing code with **fake data** when real input (a photo or an API) is not available.